Life Cycle Assessment of Exact 31-mer Counting on an HPC Node: Carbon Payback of an AI-Assisted Optimisation

Isaac Palma-Medina
Master in High Performance Computing
University of Luxembourg
isaac.palma.001@student.uni.lu

Minimum of 4 double-column pages, excluding guidance text, references and appendices.

Compiled November 10, 2026.

***Abstract*—On a shared high-performance computing cluster, a job is charged for all the hardware it reserves, used or not. This paper measures the carbon footprint of one such job, finds which resource carries it, and asks whether an AI-assisted optimisation of that resource repays its own footprint. The job counts every 31-mer in 5 000 000 DNA reads with a single-threaded Python program, on a whole 128-core node of the Aion cluster, partition `batch`. A life cycle assessment built on processor energy counters puts one run at 15 gCO_2e on that exclusive node. The idle draw of the processors carries 57 % of it, the reserved memory 30 % and the program's own work 1.6 %. An AI coding agent then changes only what surrounds the unchanged program: a request of 6 cores and 10 GiB on a shared node of the same partition, and a Cython build. With the same factors, one run is then attributed 0.84 g. The assistance and the test runs cost 18.6 g, which the saving repays in 1.32 runs. For a serial program on a shared cluster, the allocation, not the code, sets the footprint. A three-step method and a script carry this check to any batch job.**

I. Introduction

Computing draws electricity, and the carbon footprint of that electricity depends on where and how it is generated [1]. On a shared high-performance computing (HPC) cluster, a scheduler grants each job an allocation: nodes, cores and memory, held for a walltime. The hardware a job holds is attributed to it, whether its program uses that hardware or not. A request can therefore cost far more than the computation it serves.

The object of this study is one such job. It counts exactly every 31-mer, a substring of 31 letters of a DNA sequence, in 5 000 000 sequencing reads of the bacterium *Mycobacterium abscessus* (counted on 27 October 2026). Counting such substrings is a common first step in genome analysis. The counter, `kc-py1.py`, is a single-threaded Python program written for teaching [2]. The job runs on Aion, a cluster of the University of Luxembourg, where it holds a 128-core node exclusively while its program uses one core.

The scheduler reports no energy figure for this job (Section III-B). Its user sees a walltime and an exit code, but not what the run costs the climate. The allocation, more than the program, may set that cost. Correcting the allocation with an artificial intelligence (AI) coding agent has a footprint of its own, from the AI assistance and the test runs. Such a change is worth making only if each run saves enough to repay that footprint within the life of the job.

This study measures the footprint with a life cycle assessment (LCA), the standard method for the environmental impact of a product over its life [3], [4]. It counts greenhouse gases in grams of CO_2 equivalent (gCO_2e): each gas counts as the mass of CO_2 that traps as much heat over 100 years, a measure called the global warming potential over 100 years, or GWP100 [5].

This paper answers the following research question: *what is the GWP100 carbon footprint of one run of an exact 31-mer counting job on an Aion node, which resource of the job's allocation carries most of it, and how many runs does an AI-assisted optimisation of that resource need to pay back its own footprint?* The first two parts concern one particular instance; the third concerns a means [6].

The paper makes four contributions. The first is a baseline footprint of 15 gCO_2e per run, built on processor energy counters that an ordinary user can read. The second is the footprint after an AI-assisted optimisation that leaves the program unchanged: 0.84 g per run. The third is the carbon payback of that optimisation, 1.32 runs. The fourth is a three-step method, with a script, that applies the same check to any batch job. Every figure traces back to a measured value, a cited factor or a stated assumption, and the scripts that recompute them are kept with the job.

Section II reviews related work. Section III assesses the baseline job, from its scope to its hotspots. Section IV attacks those hotspots, and Section V computes the payback. Section VI discusses the threats to validity.

II. Related Work

Lannelongue et al. [1] estimate the footprint of a computation from its runtime, the cores and memory it uses, the grid intensity and the power usage effectiveness (PUE) of the data centre, in their Green Algorithms calculator. This study keeps their memory coefficient but replaces their processor term with measured energy. Cores held idle still draw power, and a model that counts only the busy cores misses it (Section III-D).

Elsworth et al. [7] measure the energy of AI serving at Google. Beside the active accelerators, they count the host machines, the idle capacity held in reserve and the data-centre overhead, which widen the energy of a median prompt 1.72-fold. Their paper is a preprint, not yet peer reviewed. This study makes the same argument for one batch job: the capacity it reserves, not the work it does, carries most of its footprint.

Lakim et al. [8] assess a language-model project end to end, small experiments included, and point to idle hardware as a reason to keep it fully used. Their count of experiments is why the payback of Section V includes the test runs, not only the AI assistance. Wadenstein and Vanderbauwhede [9] show that hardware manufacture dominates the life-cycle emissions of scientific computing centres once the grid decarbonises, which is why this study keeps a share of hardware production. Avgerinou et al. [10] find that the average PUE of EU data centres declined each year from 2009 to 2016, which is why this study takes a recent EU average [11] as a proxy for Aion.

None of these works separates, for one batch job, the energy of what it reserves from the energy of what it computes, or asks whether an AI-assisted change to that reservation repays its own cost.

III. Life Cycle Assessment

This section assesses the baseline job in the phases of ISO 14040 and 14044 [3], [4]: goal and scope, inventory, impact assessment and interpretation, followed by the reporting of its hotspots.

A. Goal and scope definition

The goal and scope fix what is studied, the unit every figure refers to, the boundary of the system and the impact method.

Goal. This study quantifies the carbon footprint of one run of the baseline job. That footprint is the reference for the optimisation of Section IV and its payback in Section V.

The job. The code is `kc-py1.py` from the public repository `lh3/kmer-cnt` [2], frozen at commit `e2574719`.[1] It reads DNA reads from standard input in FASTA, a plain-text format for sequences. DNA has two strands, and a 31-mer on one strand appears on the other as its reverse complement. The program therefore counts each 31-mer under the alphabetically smaller of its two forms, called the canonical 31-mer, and skips every 31-mer that contains `N`, the letter for an unread base. It prints a histogram of 255 lines: line n gives the number of distinct canonical 31-mers that occur n times, and line 255 also holds those that occur more often. The input is a compressed file of 187 693 955 bytes from the repository's v0.1 release [2]. The program converts lower-case bases to upper case in the last read only, so a lower-case base elsewhere would change the count. No read of this input holds one (checked on 27 October 2026). Every run writes the same 1 972-byte histogram. The footnote identifies the code, the input and the histogram, the last two by their SHA-256 checksums, fingerprints of their exact content.

Repository. The job, its optimised versions, the measurement records, the calculation scripts and the tool of Section IV-D are in the public repository https://github.com/isaac-pm/kmer-counting, at commit `85ea807999e2336d1ae21adcbf4f2cd1ef2975c3`.

The machine. The job runs in the `batch` partition of Aion, the queue for production jobs, at the `normal` quality-of-service level. A regular Aion node holds two AMD EPYC 7H12 processors of 64 cores each, 256 GB of memory, a 480 GB local solid-state disk (SSD) and one 100 Gb/s InfiniBand network port [12]. The software is Python 3.11.5 from the cluster's default release, as recorded in every run's log.

Assumption of equal nodes. Slurm, the scheduler, places each run on whichever node is free, so the runs of this study fall on several regular nodes. The study assumes that every regular node of the partition gives the same performance and the same power draw for the same request. Under this assumption, runs on different nodes measure the same job. All these nodes are of one type, but the assumption is not proven: Section VI weighs it.

Allocation and use. Each baseline run holds one node exclusively: 128 cores, all the memory Slurm can reserve (224 000 MiB, where 1 MiB is 2^{20} bytes) and a 30-minute limit. Its program uses one of those cores. The author submits it from a login node, with a standard student account. Five runs are measured, and the job is assumed to run 100 times per year (Section VI).

Functional unit. One run of `kc-py1.py` at commit `e2574719` that counts the exact canonical 31-mers of the fixed input and writes the reference histogram, submitted by a student user to partition `batch` of Aion and executed on one regular node. Its impact is reported in gCO_2e at GWP100. The allocation is deliberately not part of the unit, so that a change of allocation (Section IV) is compared on the same unit.

Method. One impact category, climate change, characterised with the IPCC 2021 method of the Intergovernmental Panel on Climate Change [5], as GWP100.

Boundary. Four components are inside the boundary: (1) the *compute node*, with its electricity during the walltime and a share of its manufacture; (2) the *interconnect*, the InfiniBand network that carries the job's reads and writes; (3) the *storage*, the shared General Parallel File System (GPFS) that holds the input, the code and the output; (4) the *site*, the cooling and power distribution around the machine, which enters through the PUE. Outside are the author's laptop, the login node, the network to the cluster, the download of the input, the development of the code and every node the allocation does not reserve. For the node, production and use are kept; for the other three components, use only. End of life is not inventoried, because the disposal route of Aion's hardware is not published.

[1] Full identifiers. Code: `lh3/kmer-cnt` commit `e2574719cfb784915d80eb5828e78dfae4cfdd7b`. Input SHA-256: `dd2ef49aebabfed1574b55fd002025f29b2fda88c70b2095f5e1b3a822cee736`. Output SHA-256: `06a32de77200121a397050e015a257953dc956b7854027a1ed73b15afc12bf42`. Job script: project repository commit `253d93ec726b82670c8f4b84da20e9528c9a59ef`.

B. Life cycle inventory

The inventory lists the physical flows the assessment rests on and how each is obtained. Table I in Appendix A gives every entry with its source and its reading date.

Time and resources. The job runs five times on 29 September 2026, one run after another on the same node. All five end without error and write the same histogram. Their walltimes span 362 to 372 s, with a median of 368 s. The scheduler's accounting records give the cores held, the memory reserved and the peak memory the program uses.

Energy. The scheduler's energy fields are empty for every run, which is missing data, not zero energy. The processors (CPUs), however, carry energy counters, called Running Average Power Limit (RAPL), that an ordinary user can read. Five more runs on the same day read them before and after the counting step, on a second regular node because the first is busy. The two processor packages draw a median of 67 834 J per run (range 67 129 to 68 827 J), or 188 W on average. Read for 60 s after the fifth run, the idle node draws 183 W. The program's own activity therefore adds only 1 863 J per run (median): 97 % of the package energy is drawn whether the program runs or not. RAPL covers the processor packages only; memory, board, disk and network card fall outside it.

Models. The Green Algorithms model [1] gives a second estimate: the walltime times the power of the cores and memory held, with 280 W per 64-core processor at its rated power [12] and 0.3725 W per GB of memory. Counting one core gives 36.7 kJ per run; counting all 128 cores at rated power gives 241 kJ. The measured packages alone, 67.8 kJ, draw almost twice the one-core figure, so both model figures serve as scenarios, not as bounds.

Compute node. The job reserves 224 000 MiB and touches 7.40 GiB of it, 3.4 %. For manufacture, the ecoinvent 3.11 database [13] holds no server dataset (searched on 29 September 2026). Its closest dataset, a 2002 desktop computer without screen, carries 221 $kgCO_2e$. A two-socket AMD EPYC server, the Dell PowerEdge R7525, has a published production footprint of 1 768.4 $kgCO_2e$, of which 854.7 kg comes from two 4 TB SSDs [14]. Neither describes an Aion node; Section III-C chooses between them. Aion has run since late 2021 and has no published retirement date [12], so a six-year service life is assumed. A shared item is divided by the node-seconds a run holds over the node-seconds the node delivers in its life: 368 s of 189 345 600 s, a share of 1.94×10^{-6}.

Interconnect and storage. The job runs on one node, so its only network traffic is file input and output. It reads about 188 MB of compressed input and 19 MiB of software, and writes the 1 972-byte histogram, as inferred from the scheduler's disk counters. Neither the network nor the file system meters its energy per job, so both are left out of the result.

Site. The PUE is the ratio of the electricity a whole data centre draws to the electricity its computing equipment draws. The centre publishes none for Aion, so the study declares 1.36, the energy-weighted average of 681 EU data centres for 2023 [11], read on 29 September 2026. This PUE adds cooling and power distribution to the counted node energy. It cannot supply the computing energy of the network and storage that is left out.

C. Life cycle impact assessment

The impact assessment multiplies each flow by a characterisation factor, an impact per unit of flow. Table II in Appendix A gives the chain, which a script of the project repository recomputes from the raw records.

Node energy. The measured packages cover the counting step only, while the job holds the whole node for its walltime. The node estimate therefore adds three terms per run: the measured package energy; the idle rate over the 1 to 3 s of the allocation outside the counting step; and the memory model, 0.3725 W/GB for all 256 GB, or 95.4 W, over the walltime. Its median is 103 146 J (0.0287 kWh, range 101 622 to 104 215 J) over 364 s, the median walltime of the same runs.

Site and grid. The PUE of 1.36 raises this to 0.0390 kWh drawn by the site. The grid factor is 337 gCO_2e/kWh: ecoinvent 3.11, market for electricity, low voltage, Luxembourg, 2020–2024, IPCC 2021 GWP100 [15], read on 19 October 2026. It describes the electricity consumed in Luxembourg, imports and grid losses included, which is the electricity the site draws. A factor for national generation alone would leave out the imports. The operational impact is $0.0390 \times 337 = 13.1$ gCO_2e per run.

Hardware. The study uses the R7525 figure, because the ecoinvent desktop models a single-processor machine of 11.3 kg from 2002. An Aion node holds one 480 GB SSD, not two 4 TB SSDs, so those are removed. This leaves 913.7 $kgCO_2e$, a proxy for a node, not its footprint. The Dell study uses IPCC 2013 and publishes no emissions per gas, so the figure cannot be converted to IPCC 2021: it is kept and labelled (Section VI). The grid factor is not applied to it, because the hardware is made worldwide. One run holds $364/189\,345\,600 = 1.92 \times 10^{-6}$ of a six-year life, so it carries 1.76 gCO_2e.

Result. One run of the baseline job emits $13.1 + 1.8 \approx 15$ gCO_2e.

D. Interpretation

Against daily life. A 5 W LED bulb lit for one hour emits 1.7 gCO_2e on the same grid factor, so one run equals about 9 hours of light. It also equals about 50 m in a small petrol car, at 291 gCO_2e per km [16], a cradle-to-grave figure with a wider boundary. At 100 runs per year, the job emits 1.5 $kgCO_2e$ per year.

Against the scenarios. Table III in Appendix A sets the chosen estimate beside the alternatives, all on the same 364 s. The measured packages alone fall 34 % short, because they lack the memory. The one-core model falls almost threefold short. It gives the processors 4.4 W where they measure 188 W, because it gives the 127 unused cores no power. The all-cores model, at the rated 560 W, overshoots by 135 kJ. The measured 188 W is 34 % of that rating, a plausible share for a node on which one core of 128 works. The program's own work above

idle, 1.9 kJ, is less than 2 % of the node estimate. A user who counts only the cores a program uses therefore understates the footprint of an exclusive job, which is why the chosen result keeps the measurement wherever one exists.

E. Reporting

The baseline footprint is 15 gCO_2e per run: 13.1 g of electricity and 1.8 g of hardware production, for one run that holds an exclusive Aion node for 364 s. It rests on IPCC 2021 GWP100, measured processor energy, modelled memory, a PUE of 1.36, 337 gCO_2e/kWh and a six-year hardware life. Across the scenarios of Table III, it ranges from 6.4 to 32 g. It scales with the node-time held, not with the work done.

Two hotspots carry the result, and both come from the allocation rather than from the counting. The idle draw of the processors takes 8.5 g, 57 % of the total with its share of the site overhead, while the program's own work takes 1.6 %. The reserved memory takes 4.4 g, 30 %, and the job touches 3.4 % of it. Hardware production takes the remaining 12 %.

IV. Optimisation

A. Optimisation procedure

The hotspots come from what the job holds, not from what it computes. The optimisation therefore changes what surrounds the program and leaves the program itself unchanged, byte for byte, so that the functional unit holds by construction. It uses two levers, in this order. (1) *Allocation.* Request only the cores and memory the program uses, on a node shared with other jobs. The job is then charged its share of the node's idle draw, memory and production, not all of it. (2) *Execution.* Run the same source faster, by compiling it with Cython, a compiler that turns Python code into C, so that the request is held for less time. This lever comes second, because the program's own work is only 1.6 % of the baseline.

Before and after keep the program, the input, the partition, the node type, the protocol (five consecutive runs, median) and the factors. They run on different nodes, so the comparison rests on the assumption of equal nodes of Section III-A.

An AI coding agent makes each change: OpenCode, with the DeepSeek V4 Flash model the course provides. The author chooses each lever, writes the agent's instructions, which forbid any change to the program, and checks every change. The agent's configuration fixes the model, a low reasoning effort, a limit on steps and the commands it may run. With a short project instruction file, it cuts the fixed cost of one request from 12 365 to 5 920 input tokens.

B. Implementation

Allocation. The new job script requests one task with 6 cores, 10 GiB of memory, no exclusive node and a 10-minute limit. The memory is 1.25 times the measured 7.40 GiB peak, rounded up. Aion grants at most 1 750 MB of memory per core (read on 27 October 2026), so 10 GiB needs 6 cores, and the program still runs on one of them. This rule comes from a failed attempt: a request of one core and 8 GB made the scheduler raise the cores silently, and the job then failed to start.

Compilation. A short build job compiles the unchanged source with Cython 3.0.10 into a standalone program, against the Python of the same software release. The build log records the checksum of the compiled source, which proves it is the frozen file. Without type annotations, Cython removes only the cost of interpreting Python, so a modest gain is expected. The compiled program runs with the same request.

Rejected. An earlier attempt by the agent, outside this scope, rewrites the counter around 62-bit integers. A rewrite changes the program, so it is discarded and left out of the payback (Section VI).

Check. Both versions write the frozen counter's histogram on a sample of 100 000 reads, and every measured run writes the reference histogram on the full input. The program, its output and the functional unit are unchanged.

C. Footprint after optimisation

Each version runs five times on 10 November 2026, alternating, on regular nodes shared with other jobs. Each run holds 6 cores and 10 GiB and peaks at 7.40 GiB, as before. The right-sized script takes a median of 367 s (362 to 377 s), against 364 s on the exclusive node: sharing the node does not slow the program. The compiled program takes 340 s (335 to 351 s), 7.4 % less.

On a shared node, the processor counters see every job, so this job's energy is attributed rather than measured. Each run is charged 6/128 of the node's idle draw, the program's own work at 5.2 W per busy core from the baseline measurement, and the memory model on the 10.74 GB reserved. Production is shared by the larger of the core and memory shares. Applied to the whole node, the same chain returns the baseline within 0.1 %.

The right-sized script is attributed 0.91 gCO_2e per run, and the compiled program 0.84 g (range 0.83 to 0.87 g), which is kept. An optimised run thus carries 6 % of the baseline, an estimated saving of 14.0 g per run. This is a smaller footprint charged to the job, under the assumption that other jobs use what it frees, not a measured fall in the cluster's emissions. The node energy charged to a run falls from 103.1 to 6.0 kJ. The idle share still leads, with 44 % of the new total, ahead of the program's own work (27 %), memory (20 %) and production (9 %). Table IV in Appendix A sets the three versions side by side. Of the 14.04 g saved, the allocation brings 13.98 g and compilation 0.07 g.

D. What generalises

1. The method. Any batch job can be checked in three steps. (1) *Recognise.* Compare what the job holds with what it uses, from the scheduler's records: the cores held against the CPU time over the walltime, and the memory reserved against the peak memory. A ratio far below one is the hotspot of this paper. (2) *Change.* Request one task with the cores the program keeps busy and 1.25 times its peak memory. Raise the cores if the cluster limits the memory per core, so that the

scheduler does not raise them silently. Drop exclusive access. Only then shorten the walltime without touching the program, for example by compiling it, and check that the gain repays the extra work. (3) *Check.* The output must be byte-identical, the job must complete within its new limits over five runs, and the footprint per run, with the same factors, must fall. The method does not help where a job already keeps its cores busy, where a cluster grants only whole nodes, or where the freed resources stay idle.

2. The tool. `allocation-footprint.py`, written by a second agent, applies the method to any finished job. From a job identifier on a cluster, or from a saved accounting record anywhere else, it prints what the job held and used, the footprint per run of that allocation and of the right-sized one, and the request lines to use. Every factor is an option, with this study's values as defaults. It needs Python 3.6 or newer and no other package, and it sits in `job/tool/` of the repository named in Section III-A, with its own README. It passes three checks. First, its inputs and outputs are those just listed. Second, it runs on jobs other than the one it is written for. It reproduces 14.88 g on a baseline record and 0.92 and 0.84 g on optimised runs, and on an unrelated 61-s measurement job it reports 2.45 g for 0.7 CPU-seconds and recommends one core and 1 GiB. Third, it stops with a clear message on a record that lacks the allocation fields and on a job that does not complete. On the optimised job, it recommends the request already in use: nothing is left to cut.

V. Carbon Payback

The payback divides the cost of producing the optimisation by the saving per run. Only the path that leads to the kept optimisation is counted; Section VI gives the weight of the discarded rewrite. Table V in Appendix A details both terms.

Cost. The AI assistance takes ten agent sessions, 62 requests and 27 924 output tokens, reasoning included. The course factor of 1.03 Wh per 1 000 output tokens [17], from one query scenario of Oviedo et al. [18], gives 0.029 kWh. As the course prescribes, the grid factor of Section III-C converts it to 9.7 gCO_2e. Where the model runs is not known, so that factor is an assumption for the AI service. The runs on Aion cost 8.87 g: 8.81 g for the ten measured runs and 0.06 g for twenty short jobs: failed launches, builds, sample tests and accounting. The total cost is 18.6 gCO_2e. Tests on the author's laptop are outside the boundary.

Payback. The saving is $14.888 - 0.844 = 14.04$ gCO_2e per run, so the payback is $18.6/14.04 = 1.32$ runs: the second run after the change repays it. At the assumed 100 runs per year, that takes about five days. The AI assistance accounts for 0.69 of these runs and the runs on Aion for 0.63. Compilation alone repays its own 7.3 g only after 109 runs, about 13 months, because it saves just 0.067 g per run. The general-purpose assistant that helped write this paper (Appendix B) is not counted, because it does not produce the optimisation.

A. What this means

One run of this job emits 15 gCO_2e on an exclusive Aion node, more than half of it from cores held idle. Without a change to the program, right-sizing and compilation cut the footprint attributed to one run to 0.84 g, and they repay their own 19 g within two runs. The lesson reaches beyond this job. For a serial program on a shared cluster, the allocation, not the code, sets the footprint. Correcting it pays back fast, because each run then saves almost all of its footprint. Making the code faster pays back slowly once the allocation is right. The next steps are to run the tool on other users' jobs, and to confirm the attributed energy with a measurement on a node that holds only the new request.

VI. Threats to Validity

This section gathers the limitations of Sections III-C to V. For each, it states whether it makes the baseline of 15 gCO_2e, the optimised 0.84 g or the payback of 1.32 runs too high or too low and, where possible, by how much. Each shift moves one input of the calculation scripts and holds the others fixed.

Energy that is not measured. No instrument reaches the whole node. The board, the local SSD, the network card and power conversion are not counted, so the result is too low. Each 50 W they draw adds 2.3 g per run, or 16 %. The memory term is a generic coefficient [1] and can err in either direction. The idle rate is a single 60-s reading, so the 57 % hotspot share moves with any change in idle draw over time. The energy of the network and the storage is left out, so the result is too low by an amount Aion does not publish.

Proxy factors. The PUE of 1.36 is an EU average, not a site value, and Aion's cold-plate cooling [12] may put the site below it. A PUE of 1.1 gives 12.4 g (17 % less), and 1.6 gives 17.2 g (16 % more). The grid factor describes low-voltage supply. If the site draws at a higher voltage, part of the grid losses does not apply, and the result is slightly too high. The dataset's description mentions high voltage although its title names low voltage; the study follows the title. The hardware proxy is a rack server, while an Aion node is one of three nodes in a blade [12]. The ecoinvent desktop would give 13.6 g (9 % less), and the R7525 with its SSDs 16.5 g (11 % more). The node's own SSD, the chassis, the switches, the storage servers and the building are left out, so the result is too low. The R7525 figure uses IPCC 2013, whose GWP100 for methane and N_2O differs from IPCC 2021 by 3 % or less [19], [5]. Each 10 % change on the hardware term moves the total by 1.2 %. A four-year life gives 15.8 g (6 % more), and an eight-year life 14.4 g (3 % less). Downtime spreads production over fewer node-seconds, so the hardware term is too low by an unknown fraction.

Cache state. File caches are neither reset nor measured. The counting step is bound by computation, with a CPU time of 99.8 % of its walltime, so a cache effect is bounded by the 2.5 % spread of the node estimates.

Equal nodes. The study relies on the assumption of equal regular nodes (Section III-A) at three points. The baseline

timing and energy come from two different nodes, the optimised runs fall on two others, and one idle reading serves every node. The runs bound the error. The baseline median is 368 s on one node and 364 s on the other; with the first, the result would be 15.05 instead of 14.89 g, 1.1 % more. An idle reading on a third node gives 178.6 W against 182.6 W, 2 % apart, which moves the idle share by the same fraction. In the optimised series, the runs on the second node differ from those on the first by less than 2 %, in opposite directions for the two versions. That is less than the 4 % spread of the runs on one node alone. The data therefore do not contradict the assumption, but two runs cannot prove it. A node 2 % slower moves the optimised result by about 2 % and the payback by less than 0.03 runs. The 7.4 % gain of compilation exceeds these differences, and the alternating runs put both versions on the same nodes.

Attributed energy after optimisation. The optimised energy is attributed, not measured. Its program term reuses the baseline's 5.2 W per busy core, although the compiled program may draw a different power. That term is 27 % of the optimised result, so an error of 20 % on it moves the result by 0.05 g and the saving by less than 0.4 %. The saving also assumes that other jobs use the 122 cores and the memory this job frees; 132 regular nodes are shared between jobs on 27 October 2026. If the freed cores stayed idle, each run would bear the whole node's idle draw: the saving would fall to 6.5 g and the payback would rise to 2.9 runs.

Cost of the AI assistance. The token factor is a ratio from one short-query scenario, 500 input and 300 output tokens to a model of more than 200 billion parameters [18], not a measurement of these sessions. Agent sessions re-send their context: these read about 11 500 input tokens per request, and very long inputs raise the energy of reading them substantially [18]. The model is a mixture of experts, with 13 billion of its 284 billion parameters active per token [20], on hardware that is not published. If the assistance cost ten times the estimate, the payback would be 7.5 runs, about four weeks. The requests reach one of three providers whose location the session records do not give. Each 100 gCO_2e/kWh more on their grid adds 0.20 runs to the payback, and twice the Luxembourg factor gives 2.0 runs.

Discarded path. The agent's discarded rewrite of the counter cost 10.1 g: 7.1 g of AI assistance and 3.0 g of runs. Counted, it would raise the payback from 1.32 to 2.04 runs.

Runs per year. The figure of 100 runs per year is an assumption. It changes the annual footprint and the payback period, not the result per run.

Scope. The result covers climate change only, for one job, and hides the mineral and water use of hardware production. End of life is outside it, so it is too low by the share of disposal, while the ecoinvent desktop proxy includes packaging and disposal. The car comparator is cradle to grave, which makes one run look smaller beside it. A change that frees the node may lead the job to run more often. The result supports no comparison with another job.

AI Usage Transparency

Two AI models assist this work. Claude Opus 5.5, used in chat and through the Claude Code agent, guides the setting up of the job and its measurements, runs the measurement campaign of Section IV, writes calculation scripts, checks facts and references, and edits and proofreads the text of every section. DeepSeek V4 Flash, used through the OpenCode agent, writes the job scripts, the build and the tool of Section IV, and no text of this paper. Every output kept is checked against the measurements or against a source the author reads, and the author is responsible for this paper and for the script of Section IV-D. Appendix B estimates this assistance at 47.7 gCO_2e.

References

[1] L. Lannelongue, J. Grealey, and M. Inouye, "Green Algorithms: Quantifying the carbon footprint of computation," *Advanced Science*, vol. 8, no. 12, p. 2100707, 2021. [Online]. Available: https://doi.org/10.1002/advs.202100707

[2] H. Li, "kmer-cnt: Code examples of fast and simple k-mer counters for tutorial purposes," https://github.com/lh3/kmer-cnt/tree/e2574719cfb784915d80eb5828e78dfae4cfdd7b, 2020, commit e2574719cfb784915d80eb5828e78dfae4cfdd7b, MIT licence. Accessed 28 September 2026.

[3] *ISO 14040:2006, Environmental management: Life cycle assessment: Principles and framework*, International Organization for Standardization Std., 2006.

[4] *ISO 14044:2006, Environmental management: Life cycle assessment: Requirements and guidelines*, International Organization for Standardization Std., 2006.

[5] P. Forster, T. Storelvmo, K. Armour, W. Collins, J.-L. Dufresne, D. Frame, D. J. Lunt, T. Mauritsen, M. D. Palmer, M. Watanabe, M. Wild, and H. Zhang, "The Earth's energy budget, climate feedbacks, and climate sensitivity," in *Climate Change 2021: The Physical Science Basis. Contribution of Working Group I to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change*. Cambridge, UK and New York, NY, USA: Cambridge University Press, 2021, pp. 923–1054, section 7.6, Table 7.15: GWP100 of fossil CH_4 29.8 and of N_2O 273, including climate–carbon feedbacks. [Online]. Available: https://doi.org/10.1017/9781009157896.009

[6] M. Shaw, "Writing good software engineering research papers," in *Proceedings of the 25th International Conference on Software Engineering (ICSE 2003)*. IEEE Computer Society, 2003, pp. 726–736. [Online]. Available: https://doi.org/10.1109/ICSE.2003.1201262

[7] C. Elsworth, K. Huang, D. Patterson, I. Schneider, R. Sedivy, S. Goodman, B. Townsend, P. Ranganathan, J. Dean, A. Vahdat, B. Gomes, and J. Manyika, "Measuring the environmental impact of delivering AI at Google scale," arXiv preprint arXiv:2508.15734, not peer reviewed, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2508.15734

[8] I. Lakim, E. Almazrouei, I. Abualhaol, M. Debbah, and J. Launay, "A holistic assessment of the carbon footprint of Noor, a very large Arabic language model," in *Proceedings of BigScience Episode #5 – Workshop on Challenges & Perspectives in Creating Large Language Models*. Association for Computational Linguistics, 2022, pp. 84–94. [Online]. Available: https://doi.org/10.18653/v1/2022.bigscience-1.8

[9] M. Wadenstein and W. Vanderbauwhede, "Life cycle analysis for emissions of scientific computing centres," *The European Physical Journal C*, vol. 85, no. 8, p. 913, 2025. [Online]. Available: https://doi.org/10.1140/epjc/s10052-025-14650-8

[10] M. Avgerinou, P. Bertoldi, and L. Castellazzi, "Trends in data centre energy consumption under the European code of conduct for data centre energy efficiency," *Energies*, vol. 10, no. 10, p. 1470, 2017. [Online]. Available: https://doi.org/10.3390/en10101470

[11] D. Heatubun, I. Dragan, M. Bolchi, P. Lopes Bautista, S. Hinterholzer, R. Hintemann, N. Marx, and P. Furda, "Assessment of the energy performance and sustainability of data centres in EU: First technical report," European Commission, Directorate-General for Energy; Publications Office of the European Union, Luxembourg, Tech. Rep.,

Jul. 2025, prepared by EY with Borderstep and AIT. Figure 6, p. 34: EU average PUE 1.36 for 2023 data, weighted by total energy consumption over the 681 reporting data centres. Accessed 29 September 2026. [Online]. Available: https://doi.org/10.2833/3168794

[12] ULHPC Team, "Aion: compute nodes, interconnect, timeline, and GPFS file system," ULHPC Technical Documentation, https://hpc-docs.uni.lu/systems/aion/compute/, https://hpc-docs.uni.lu/systems/aion/timeline/, https://hpc-docs.uni.lu/filesystems/gpfs/, 2026, University of Luxembourg. Accessed 29 September 2026.

[13] ecoinvent Association, "ecoinvent database version 3.11, system model allocation, cut-off by classification," ecoQuery, https://ecoquery.ecoinvent.org/3.11/cutoff/dataset/3193/impact_assessment, 2024, dataset: computer production, desktop, without screen (GLO). Released 20 November 2024. Accessed 29 September 2026.

[14] Sphera Solutions, "Life cycle assessment – Dell servers R6515, R7515, R6525, R7525," Prepared on behalf of Dell Technologies, Report v1.0, Feb. 2021, Table 4-5, p. 47. Accessed 29 September 2026. [Online]. Available: https://www.delltechnologies.com/asset/en-us/products/servers/technical-support/full-lca-of-dell-severs-r6515-r7515-r6525-r7525.pdf

[15] ecoinvent Association, "ecoinvent database version 3.11, system model allocation, cut-off by classification: market for electricity, low voltage, LU," ecoQuery, https://ecoquery.ecoinvent.org/3.11/cutoff/dataset/2297/impact_assessment, 2024, time period 2020–2024. IPCC 2021, climate change: total (excl. biogenic CO_2), GWP100: 0.337 kg CO_2e per kWh. Accessed 19 October 2026.

[16] B. Ries, "Green IT, session 2: Sustainable development," Lecture slides, MICS2-63, University of Luxembourg, Sep. 2026, slide 25 and reference [2]: ecoinvent 3.11 cut-off, transport, passenger, car, petrol, small size, EURO 5, RER, 0.291 kg CO_2e per vehicle-km at GWP100, cradle to grave.

[17] ——, "Green IT, session 8: AI-assisted optimisation and your carbon payback," Lecture slides, MICS2-63, University of Luxembourg, Oct. 2026, slide 11: course factor of 1.03 Wh per 1 000 output tokens, from Oviedo et al., Joule 2026, "an estimate for short queries".

[18] F. Oviedo, F. Kazhamiaka, E. Choukse, A. Kim, A. Luers, M. Nakagawa *et al.*, "Energy use of AI inference, efficiency pathways, and test-time scaling," *Joule*, vol. 10, no. 8, p. 102430, 2026, read in the author version, arXiv:2509.20241v2: 0.31 Wh per standard query of 300 median output tokens and 500 input tokens, models above 200B parameters on H100 nodes, that is 1.03 Wh per 1 000 output tokens. Accessed 27 October 2026. [Online]. Available: https://doi.org/10.1016/j.joule.2026.102430

[19] G. Myhre, D. Shindell, F.-M. Bréon, W. Collins, J. Fuglestvedt, J. Huang, D. Koch, J.-F. Lamarque, D. Lee, B. Mendoza, T. Nakajima, A. Robock, G. Stephens, T. Takemura, and H. Zhang, "Anthropogenic and natural radiative forcing," in *Climate Change 2013: The Physical Science Basis. Contribution of Working Group I to the Fifth Assessment Report of the Intergovernmental Panel on Climate Change*. Cambridge, UK and New York, NY, USA: Cambridge University Press, 2013, pp. 659–740, table 8.7, GWP100 without climate–carbon feedbacks: CH_4 28 and N_2O 265; fossil CH_4 30 with the CO_2 from its oxidation (note to Table 8.7, Table 8.A.1). [Online]. Available: https://doi.org/10.1017/CBO9781107415324.018

[20] OpenRouter, "DeepSeek V4 Flash 0731 (`deepseek/deepseek-v4-flash-0731`)," Model listing, 2026, sparse mixture of experts, 13B active parameters out of 284B total. Read through the models API on 28 October 2026. [Online]. Available: https://openrouter.ai/deepseek/deepseek-v4-flash-0731

APPENDIX A
FIGURES AND TABLES

TABLE I

LIFE CYCLE INVENTORY OF ONE RUN (SECTION III-B). MEASURED ROWS ARE THE MEDIAN OF FIVE RUNS ON `AION-0127`, EXCEPT THE RAPL ROWS (`AION-0144`); ALL ROWS READ ON 29 SEPTEMBER 2026 UNLESS STATED.

Component	Quantity	Value and unit	Source	Kind
Compute node	Walltime	368 s (range 362–372)	`sacct Elapsed`, jobs 15860349–53	measured
	Nodes, cores held	1 node, 128 cores	`sacct AllocNodes, AllocCPUS`	measured
	Memory reserved	224 000 MiB	`sacct ReqMem`	measured
	Memory touched	7 760 436 KiB (7.40 GiB)	`sacct MaxRSS`, step `.0`	measured
	Electricity, scheduler	none returned	`ConsumedEnergy(Raw)` empty	measured
	CPU-package energy per run	67 834 J (range 67 129–68 827)	RAPL, jobs 15861652–60	measured
	CPU-package power, idle	183 W (10 959 J in 60 s)	RAPL, job 15861674	measured
	Package energy above idle	1 863 J (range 1 690–1 953)	run − idle rate × step time	derived
	Electricity, 1 core active	36.7 kJ	Green Algorithms [1]	model
	Electricity, 128 cores at TDP	241 kJ	Green Algorithms [1]	model
	Hardware	2 × EPYC 7H12, 256 GB	[12], `lscpu`	looked up
	Manufacture, ecoinvent proxy	221 $kgCO_2e$ per unit	[13], desktop without screen	looked up
	Manufacture, vendor proxy	1 768.4 $kgCO_2e$ per unit	[14], R7525, IPCC 2013	looked up
	Service life	6 years	Section VI	assumption
	Share per run	1.94×10^{-6}	368 s / 189 345 600 s (Table II: 364 s)	derived
Interconnect	Bytes read from storage	≈187.7 MB + 19 MiB	`MaxDiskRead`, batch minus pipe	inferred
	Bytes written to storage	1 972 B + log	`stat` of output	measured
Storage	Input stored	187 693 955 B	`stat`, SHA-256 checked (28 Sep)	measured
	Output stored	1 972 B	`stat`	measured
Site	PUE	1.36	EU average, 2023 data [11]	declared proxy
All	Runs per year	100	Section VI	assumption
	Node performance	equal on every regular node	Sections III-A, VI	assumption

TABLE II

IMPACT CHAIN FOR ONE RUN (SECTION III-C). ENERGY VALUES ARE THE MEDIANS OF THE FIVE RUNS ON `AION-0144` (JOBS 15861652–60), EACH PAIRED WITH ITS OWN `ELAPSED`; RECOMPUTED BY `LCIA-CALCULATION.PY` ON 19 OCTOBER 2026. IPCC 2021 GWP100 UNLESS STATED.

Step	Value and unit	Source or rule	Kind
Walltime held (`Elapsed`)	364 s (range 359–368)	`sacct`, jobs 15861652–60	measured
CPU packages, Python step	67 834 J	RAPL	measured
CPU packages, rest of allocation	0.2–0.6 kJ	182.6 W idle × (`Elapsed` − step)	derived
Memory	95.4 W × 364 s = 34.7 kJ	0.3725 W/GB × 256 GB [1]	model
Node estimate	103 146 J = 0.0287 kWh	sum of the three, per run	derived
Site electricity	0.0390 kWh	× PUE 1.36 [11]	declared proxy
Grid factor	337 gCO_2e/kWh	ecoinvent 3.11, electricity low voltage, LU [15], read 19 Oct 2026	looked up
Operational	**13.1 gCO_2e**	0.0390 × 337	derived
Hardware production	913.7 $kgCO_2e$ (IPCC 2013)	R7525 1 768.4 kg − 854.7 kg SSDs [14]	looked up
Share per run	1.92×10^{-6}	364 s / 189 345 600 s (six years)	assumption
Embodied	**1.76 gCO_2e**	913.7 kg × 1.92×10^{-6}	derived
Total	**15 gCO_2e per run**	13.1 + 1.8, two significant figures	derived

TABLE III

ENERGY SCENARIOS FOR ONE RUN (SECTION III-D), ALL ON THE 364 S MEDIAN `ELAPSED` OF `AION-0144`, WITH PUE 1.36, 337 gCO_2e/KWH AND THE CHOSEN 1.76 G OF HARDWARE PRODUCTION. THE GREEN ALGORITHMS ROWS USE 4.375 W PER CORE (280 W PER 64-CORE PROCESSOR [12]) AND 0.3725 W/GB FOR 256 GB [1].

Scenario	Node energy (kJ)	Site (kWh)	Operational (gCO_2e)	Total (gCO_2e)
CPU packages, Python step only (Section III-B)	67.8	0.0256	8.6	10.4
Chosen: packages over the allocation + memory model	**103.1**	**0.0390**	**13.1**	**14.9**
Green Algorithms, 1 core + 256 GB	36.3	0.0137	4.6	6.4
Green Algorithms, 128 cores at rated power + 256 GB	238.6	0.0901	30.4	32.1
CPU packages above idle, Python step (the program's own work)	1.9	0.0007	0.24	2.0

TABLE IV

ONE RUN BEFORE AND AFTER OPTIMISATION (SECTION IV-C). BEFORE: MEDIAN OF THE FIVE RAPL RUNS ON `AION-0144`, EXCLUSIVE NODE. AFTER: MEDIANS OF FIVE RUNS EACH ON 10 NOVEMBER 2026 ON SHARED REGULAR NODES (RIGHT-SIZED: JOBS 16225119, 23, 27, 31, 35; CYTHON: JOBS 16225121, 25, 29, 33, 37), THE SAME UNCHANGED PROGRAM, ENERGY ATTRIBUTED BY SHARE OF NODE; COMPUTED BY `OPTIMISED-CALCULATION.PY`. SAME FACTORS THROUGHOUT: PUE 1.36, 337 gCO_2E/kWh, 913.7 $kgCO_2E$ PER NODE OVER SIX YEARS.

Quantity	Before	Right-sized	+ Cython	Source or rule
Cores held	128	6	6	`sacct AllocCPUS`
Memory reserved	256 GB (whole node)	10.74 GB	10.74 GB	`sacct ReqMem` (10 GiB)
Walltime (`Elapsed`)	364 s	367 s (362–377)	340 s (335–351)	`sacct`
Peak memory of the program	7.40 GiB	7.40 GiB	7.40 GiB	`sacct MaxRSS`
Idle package draw charged	66.6 kJ	3.1 kJ	2.9 kJ	182.6 W × cores/128 × walltime
Program's own work	1.9 kJ	1.9 kJ	1.8 kJ	5.2 W per busy core × CPU time
Memory	34.7 kJ	1.5 kJ	1.4 kJ	0.3725 W/GB × GB reserved × walltime
Node energy	103.1 kJ	6.5 kJ	6.0 kJ	sum; before measured with RAPL, after attributed
Operational	**13.13 g**	**0.828 g**	**0.768 g**	× PUE × grid
Embodied	**1.76 g**	**0.083 g**	**0.077 g**	by the larger of core and memory share
Total, gCO_2e per run	**14.89**	**0.911**	**0.844**	saving 14.04 g (94 %) with Cython

TABLE V

CARBON PAYBACK OF THE OPTIMISATION (SECTION V), COMPUTED ON 10 NOVEMBER 2026 BY `OPENCODE-USAGE.PY` AND `PAYBACK-CALCULATION.PY` FROM THE EXPORTED OPENCODE SESSIONS AND THE AION ACCOUNTING RECORDS.

Item	gCO_2e	Source or rule
AI assistance: 10 OpenCode sessions, 62 requests, 27 924 output tokens	9.69	1.03 Wh/1 000 output tokens [17], [18] × 337 g/kWh, assumed
Ten measured runs, five per variant	8.81	Table IV chain, per run
Seven short jobs (launch failures, builds, sample tests), 13 accounting jobs	0.06	same chain, 2–12 s each
Cost of producing the optimisation	**18.57**	sum, kept path only
Saving per run	14.04	$14.888 - 0.844$
Payback	**1.32 runs**	cost / saving; about 5 days at 100 runs per year
Discarded counter rewrite, not counted	10.10	session 7.06 g + five runs 3.04 g; payback 2.04 runs if counted

APPENDIX B
AI USAGE JOURNAL

AI model: Claude Opus 5.5 (`claude-opus-5-5`)
Interaction: chat, through the Claude desktop app
Prompts: 17
Input tokens: 13 418 158, mostly re-read context
Output tokens: 88 944
Energy factor: 1.03 Wh per 1000 output tokens, so 91.6 Wh
Grid factor: 337 gCO_2e per kWh, ecoinvent 3.11, Luxembourg low-voltage market, read on 19 October 2026
Estimated: 30.9 gCO_2e
Comparable to: 18.3 hours of a 5 W LED, or 106 m in a small petrol car [16]

Kinds of task, and which sections they touched: I used the chat as a guide while I set up the repository and the job, and then for general corrections and fact checking of Sections III.A and III.B and of the references. All the text was spell-checked and proofread with it. The choices of scope, boundary and measurement were mine, taken step by step with the chat.

Wrong information:

1) it split the commit hash of the job wrongly when breaking it over lines. I compared it with the repository and corrected it.
2) it described the end-of-life treatment of the cut-off model wrongly. I rewrote the sentence.
3) some reference details were wrong: the release year of ecoinvent 3.11 and the data year of the EU PUE. I checked each reference at its source.

AI model: DeepSeek V4 Flash, pinned as `deepseek/deepseek-v4-flash-0731` through OpenRouter
Interaction: agentic, through OpenCode with the course configuration: version 1.19.3 on 27 October 2026 from 20:03 to 20:40 UTC (sessions 1–8), version 2.2.0 on 10 November 2026 from 15:56 to 16:02 UTC (sessions 9–11)
Prompts: 14, in 11 sessions, answered in 92 model requests
Input tokens: 1 299 052 (183 148 new, 1 115 904 read from the provider's cache)
Output tokens: 48 256 (25 803 of answer, 22 453 of reasoning)
Energy factor: 1.03 Wh per 1000 output tokens, the course factor [17], [18], reasoning counted as output, so 49.7 Wh
Grid factor: 337 gCO_2e per kWh, ecoinvent 3.11, Luxembourg low-voltage market, read on 19 October 2026
Estimated: 16.8 gCO_2e in all. Of this, 9.7 g is the kept path, the AI part of the cost in Section V, and 7.1 g is the discarded counter of session 4
Comparable to: 9.9 hours of a 5 W LED, or 58 m in a small petrol car [16]
Source: the eleven sessions exported from OpenCode and summed by a script, both in the project repository; OpenRouter billed USD 0.081.

Configuration: before the first task, OpenCode was set up to spend few tokens, as Sessions 7 and 8 advise. Three agent files fix the model, a low reasoning effort, at most 30 steps per request (10 for the reader), no web access and a short list of allowed shell commands, with no `ssh`, `sbatch` or `git`: an *optimiser* for the job, a *builder* for the tool and a *reader* that only reads. A five-line project instruction file replaces the long one OpenCode would otherwise load. The fixed cost of a one-word request falls from 12 365 input tokens to 5 920 (optimiser) and 4 786 (reader). Every task prompt names the input, the objective, the constraints from the LCA and the expected output. The agent cannot reach Aion: every run there was submitted and collected outside OpenCode. I entered the prompts of sessions 1–8 directly in the OpenCode TUI. Sessions 9–11 were started from the command line (`opencode run`) by Claude Code, at my request, with the prompts listed below. I checked every output before use.

Sessions, prompts and what the agent did:

1) *Smoke test, default agent.* Prompt: "Reply with exactly one word: ok". It answers "ok". 12 365 input tokens, 17 output, 0.006 g.
2) *Smoke test, reader agent.* Same prompt and answer. 4 786 input tokens, 2 output, 0.001 g.
3) *Smoke test, optimiser agent.* Same prompt and answer. 5 920 input tokens, 14 output, 0.005 g.
4) *Counter (optimiser).* Prompt: write `kc-py1-opt.py`, a faster, lower-memory replacement for the frozen counter, with the same input, a byte-identical histogram, one thread and only the standard library; test it against the frozen counter on a 20 000-line sample and report both timings. It reads the frozen counter and job script, writes the new counter (each 31-mer as a 62-bit integer, updated base by base), finds and fixes a bug that left stale bits in the integer, and shows an empty difference on the sample (0.46 to 0.48 s against 0.55 to 0.73 s). It stops at the 30-step limit. 30 requests, 20 332 output tokens, 7.06 g. (IV.B)
5) *Job script (optimiser).* Prompt: write `run-optimised.sh` from the frozen Slurm script, with the same input, modules and logging, but one task, one core, no exclusive node, memory sized to the measured 5.65 GiB peak plus a margin and a time limit that fits a slow node; report each `#SBATCH` change and the hotspot it attacks. It requests 1 core, 8 GB and 10 minutes, drops exclusive access and tables the changes. 4 requests, 1 537 output tokens, 0.53 g. (IV.B)
6) *Tool (builder), plus one follow-up.* Prompt: write `allocation-footprint.py` and its README; from a saved `sacct` record or a job number, print what the job held and used, the gCO_2e of that allocation and of a right-sized one, and the `#SBATCH` lines to use; every factor an option with this study's values as defaults; a clear error on missing fields or an unfinished job; check it on a baseline record, on the idle measurement and on the probe record that lacks fields. Follow-up: print `--ntasks=1` and `--cpus-per-task` instead of `--ntasks-per-node`. It writes both files, runs the three checks (the probe fails cleanly) and makes the fix.

15 requests, 11 286 output tokens, 3.92 g. (IV.D)

7) *Launch fix (optimiser), plus one follow-up, for the script of session 4's counter.* Prompt: the three runs on Aion fail in 2 s with an `srun` error, because the batch partition grants at most 1 750 MB per core and raises the 8 GB request to 5 cores; make the request consistent, with as few cores as that allows, a safe margin above the peak and one task. It requests 4 cores and 6 GB. Follow-up: 6 GB leaves only 6 % above the peak, so use 6 800 MB, still within 4 cores. It makes the change. 5 requests, 2 097 output tokens, 0.73 g. (IV.B)
8) *Python 3.6 fix (builder).* Prompt: on the Aion login node, whose Python is 3.6.8, the tool fails on a keyword that version lacks; make the whole script run on 3.6, correct the README's units, and rerun the baseline check. It replaces the keyword, searches the script for other features newer than 3.6 (none), corrects the README and reruns the check. 10 requests, 2 773 output tokens, 0.96 g. (IV.D)
9) *Right-sized script (optimiser), 10 November.* After the tutoring session fixed the scope to everything around the code, I rewrote the optimiser's instructions to forbid any change to the program. Prompt: write `run-rightsized.sh`, which runs the unchanged `kc-py1.py` with one task, no exclusive node, memory of 1.25 times the 7.40 GiB peak rounded up to whole GiB, the fewest cores whose 1 750 MB each cover it, and a 10-minute limit. It requests 6 cores and 10 GiB and shows the arithmetic. 4 requests, 1 427 output tokens, 0.50 g. (IV.B)
10) *Cython build (optimiser).* Prompt: compile the unchanged source with Cython into a standalone executable in a short build job that prints the SHA-256 of the source and runs a one-read test, and write `run-cython.sh` with the request of `run-rightsized.sh`. It writes both. 15 requests, 7 307 output tokens, 2.54 g. (IV.B)
11) *Build fix (optimiser).* Prompt: the build fails because Cython rejects `kc-py1` as a module name; fix it without touching the source. It adds an explicit module name. 6 requests, 1 464 output tokens, 0.51 g. (IV.B)

Session 4's counter changes the program, which the scope of the study excludes. It is kept only as a record, its five runs are discarded, and the payback leaves out both (Section VI). Sessions 5 and 7 wrote the job script for that counter; their lesson on Aion's memory limit per core is what the request of session 9 follows. The follow-ups of sessions 6 and 7 ran under OpenCode's default agent, because those sessions were resumed without naming their agent. The exports show the same pinned model for all 92 requests, but the step limit and the list of allowed commands did not apply to those 7 requests. Two attempts spent no tokens and are not counted: a first run that waited for keyboard input without starting a session, and a session-title request that OpenCode sends to another model by default, which found no provider and failed.

Kinds of task, and which sections they touched: writing code only: the right-sized job script, the Cython build and its job script (IV.B, measured in IV.C), a discarded rewrite of the counter (IV.B) and the tool that carries the method to other jobs (IV.D). The tokens of every session except session 4 are the AI part of the cost in Section V. It wrote no text of this paper, found no reference and ran nothing on Aion.

Wrong information:

1) it stated that its rewritten counter peaks "well under 1 GiB" on the full input. Its test covered only a 20 000-line sample. A full run under `/usr/bin/time` measured 5.65 GiB, and the memory request of that counter's job script rested on that figure.
2) its first job script asked for 8 GB on one core. Neither the prompt nor the script allowed for Aion's limit of 1 750 MB per core, so the scheduler raised the job to 5 cores and the three runs failed within 3 s. I found the error in the job logs and in `sacct`, and read the limit with `scontrol`.
3) it estimated 2 to 4 gCO_2e per run after optimisation, with no calculation behind it. Its five runs, attributed with the chain of Section IV-C, gave 0.61 g. I did not use the estimate.
4) it called 6 GB "comfortably above" the 5.65 GiB peak. That is a 6 % margin, for a peak measured with a different Python build than Aion's. I rejected it before any run and asked for 6 800 MB.
5) the tool printed `--ntasks-per-node=N`, which starts N copies of a single-threaded program. I saw it when reading the tool's output and asked for the fix.
6) the tool used a Python feature that the Aion login node's Python 3.6 lacks, so it crashed there. I found it when running the tool live on Aion.
7) its README called the 913.7 kg production footprint a hardware mass, and gave the memory units as GiB instead of GB. I found both by comparing the README with my source register.
8) its Cython build script named the module after the file, which Cython rejects because of the hyphen, so the first build failed within 3 s. I found the error in the build log.
9) it described the Cython executable as having "no interpreter startup". The executable embeds and starts the same Python interpreter. I checked it against the build command and did not use the claim.
10) after the build fix, it gave the SHA-256 of the histogram as the expected SHA-256 of the source file. I compared it with the hash of the frozen source in the build log.